IT 301 – OBJECT-ORIENTED PROGRAMMING

Hands-On Laboratory

Build Your Own C# Console Program

Item	Details
Course	IT 301 – Object-Oriented Programming
Platform	C# Console Application (.NET)
Total Points	100
Work Type	Individual
Name / Section	_____ / _____

1. Laboratory Overview

In this laboratory, you will design and build **your own original C# console program** that brings together the concepts you have learned so far. There is no assigned topic. You decide what your program does, who it is for, and how it works. The goal is to show that you can combine the core building blocks of programming into one working, well-structured application.

Your program must be **your own idea and your own code**. Programs copied or adapted from the internet, classmates, tutorials, or AI-generated solutions will not be accepted.

2. Learning Objectives

After completing this laboratory, you should be able to:

1. Declare and use variables with appropriate data types and meaningful names.
2. Store and process collections of data using arrays.
3. Control program flow using conditional statements.
4. Repeat tasks efficiently using looping statements.
5. Apply arithmetic, relational, logical, assignment, and increment/decrement operators correctly.
6. Validate user input so the program does not accept bad data.
7. Handle runtime errors using exception handling.
8. Plan, design, test, and explain an original program.

3. Instructions

Step 1: Come up with your own idea

Think of a small, practical, or creative program that you would actually find useful or fun. You should be able to explain it in two or three sentences. Ask yourself:

- What problem does my program solve, or what experience does it give the user?
- What information will the user enter?

- What will the program calculate, decide, or display?
- Where would an array be the natural way to store my data?

Your idea should come from **your own experience, interests, school life, hobbies, home, community, or imagination**. Avoid the first idea that every classmate might think of, and do not search the internet for ideas to copy.

Step 2: Plan before you code

Before opening Visual Studio, write a short **Program Proposal** (see Section 6) that includes your program title, purpose, inputs, processes, and outputs. A simple flowchart or pseudocode is encouraged.

Step 3: Build the program

Create a new **Console App** project and write your program. Build it in stages: get one feature working, test it, then add the next.

Step 4: Test and improve

Try to break your own program. Enter wrong values, empty input, letters instead of numbers, negative numbers, and extremely large numbers. Fix any problem you find.

Step 5: Document and submit

Add comments to your code, prepare your submission files, and be ready to demonstrate and explain your program.

4. Required Program Components

Your program **must include every item below**. Each one is graded separately.

#	Required Component	What Must Be Shown
1	**Variables and data types**	Properly declared variables using at least four different data types (e.g., `int`, `double`, `string`, `bool`, `char`, `decimal`). Use meaningful names and follow C# naming conventions. Use `const` for at least one fixed value.
2	**Arrays**	At least one array that is actually used by your program's logic (e.g., to store, search, sort, total, or display values). A one-dimensional array is the minimum; a two-dimensional array earns recognition for extra effort.
3	**Conditional statements**	Use of `if`, `else if`, and `else`. Use `switch` at least once (e.g., for a menu).
4	**Looping statements**	Use at least two different kinds of loops among `for`, `while`, `do-while`, and `foreach`. One loop should keep the program running until the user chooses to exit.
5	**Operators**	Arithmetic (`+ - * / %`), relational (`== != < > <= >=`), logical (`&& || !`), assignment (`= += -= *= /=`), and increment/decrement (`++ --`). Each type must appear and serve a purpose.

#	Required Component	What Must Be Shown
6	**Data validation**	Every user input must be checked before it is used. Use `TryParse`, range checks, empty-string checks, and loops that re-prompt the user until the input is valid.
7	**Exception handling**	Use `try`, `catch`, and `finally`. Catch at least two specific exception types (e.g., `FormatException`, `DivideByZeroException`, `IndexOutOfRangeException`, `OverflowException`) in addition to a general `Exception`.

5. Program Requirements and Rules

Functionality

- The program must compile and run without errors.
- It must have a clear **welcome message** and a **menu or flow** that is easy to follow.
- It must process data and produce **meaningful output**, not just print what the user typed.
- It must not crash on invalid input.
- The user must be able to exit the program properly.

Code Quality

- Use meaningful names for variables, such as `studentGrades` and not `x1`.
- Use consistent indentation and spacing.
- Add comments that explain the purpose of each major section (not every single line).
- Group related code logically. Using methods to organize your code is encouraged.

Scope

- Stay within the topics covered in class. Using classes, objects, or methods is allowed and welcome if you are comfortable with them, but they are not required.
- Do not use external libraries, NuGet packages, or advanced features you cannot explain.

6. Deliverables

Submit the following:

1. **Program Proposal (one page).** Name, section, program title, short description of the purpose, list of inputs, processes, and outputs, and a flowchart or pseudocode.
2. **Source code.** The complete project folder (zipped) or the `Program.cs` file, with your name, section, and date in a comment at the top.
3. **Screenshots.** At least five screenshots showing normal use, invalid input being rejected, an exception being handled, and the exit of the program.
4. **Google Classroom.** The completed tasks and documentation must be submitted within the given time frame.
5. **Originality Statement.** A short signed statement (see Section 9).

7. Grading Rubric (100 Points)

Criteria	Excellent	Satisfactory	Needs Improvement	Pts
Originality and concept	Creative, clearly original idea with a clear purpose; student can explain how they came up with it	Idea is workable but common or only partly developed	Idea is unclear, copied, or too trivial	10
Variables and data types	Four or more types used correctly, meaningful names, `const` used	Some types used correctly; naming is inconsistent	Few or incorrect declarations; poor naming	10
Arrays	Arrays are central to the logic and used correctly	Arrays are present but used in a limited way	Array is missing, unused, or incorrect	10
Conditional statements	`if`/`else if`/`else` and `switch` used correctly and meaningfully	Conditionals used but limited or partly incorrect	Conditionals missing or faulty	10
Looping statements	Two or more loop types used correctly; main program loop works	One loop type used correctly	Loops missing, accidentally infinite, or incorrect	10
Operators	All five operator categories appear and serve a real purpose	Three or four categories used	Two or fewer categories, or used incorrectly	15
Data validation	All inputs validated; user re-prompted until correct; never accepts bad data	Most inputs validated	Little or no validation	10
Exception handling	`try`/`catch`/`finally` with two or more specific exceptions and clear messages	Basic `try`/`catch` used	Missing or ineffective	10
Code quality and documentation	Clean, readable, well-commented	Mostly readable, some comments	Hard to read, no comments	5
Demonstration and explanation	Runs smoothly; student explains the code confidently	Minor issues; explanation partly clear	Fails to run or student cannot explain the code	10
TOTAL				**100**

Deductions

- **Late submission:** as stated in the course policy.
- **Missing required component:** no points for that criterion.
- **Program that does not compile:** maximum of 50 points for the entire laboratory.
- **Plagiarism or copied code:** zero (0) for the laboratory and subject to disciplinary action under school policy.

8. Tips for Success

- Start small. A simple program that works is better than an ambitious one that crashes.
- Build and test one feature at a time.
- Keep a list of the test inputs you tried, including the ones designed to break your program.
- Read your error messages. They usually tell you exactly which line has the problem.
- Be ready to explain **every line** of your code. If you cannot explain it, you should not submit it.

9. Academic Integrity and Originality

This laboratory measures **your** understanding. By submitting your work, you confirm that:

- The program idea is your own.
- You wrote the code yourself.
- You did not copy complete or partial programs from websites, repositories, classmates, videos, or AI tools.
- You may consult official documentation or your class notes for syntax, but the logic and structure are yours.

Your instructor may ask you to explain, modify, or re-create parts of your code during the demonstration. Inability to explain your own code will be treated as evidence of copying and will affect your grade.

Originality Statement

> I, _____, certify that the program I am submitting for the IT 301 hands-on laboratory is my own original work and idea, and that I did not copy it from any other source.

Signature: _____ Date: _____

10. Submission Checklist

- ☐ Program Proposal with flowchart or pseudocode
- ☐ Complete source code with name, section, and date in a header comment
- ☐ Program compiles and runs without errors
- ☐ All seven required components are present
- ☐ Five or more screenshots
- ☐ Signed Originality Statement

☐ Ready for live demonstration